

[NLP & Text Mining]

텍스트 전처리

박성호

컴퓨터공학부

한국공학대학교

학습 목표

주요 용어들 확인

텍스트를 전처리하기 위한 각종 기법에 대해서 배웁니다.

- 토큰화 (Tokenization)
- 품사 태깅 (Part of speech tagging)
- 정규화 (Normalization) – stemming, Lemmatization
- 정규 표현식 (Regular Expression)

주요 용어들

- **말뭉치 (corpus):** 수집된 텍스트 데이터
- **문장 (Sentence) :**
 - 생각이나 감정을 말과 글로 표현할 때 완결된 내용을 나타내는 최소의 독립적인 형식 단위.
 - . 마침표, !느낌표, ? 물음표와 같은 기호로 구분됨
- **단락 (Paragraph):** 정보를 공유하는 문장 집합을 표현
- **문서 (Document):** 단락의 집합
- **토큰 (Token):** 가장 작은 단위, 문맥에 따라서, 단어 (word), 형태소 (morpheme), 서브워드 (subword).

```
bot[4]:
```

	id	document	label
0	8112052	어릴때보고 지금다시봐도 재밌어요ㅋㅋ	1
1	8132799	디자인을 배우는 학생으로, 외국디자인과 그들이 일군 전통을 통해 발전해가는 문화산...	1
2	4655635	플리스스토리 시리즈는 1부터 뉴까지 버릴게 하나도 없음.. 최고.	1
3	9251303	와.. 연기가 진짜 개쩔구나.. 지루할거라고 생각했는데 몰입해서 봤다.. 그대 이런...	1
4	10067386	안개 자욱한 밤하늘에 떠 있는 초승달 같은 영화.	1

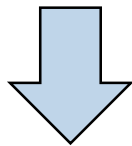
토큰화 (Tokenization)

- 토큰(Token)이라는 단위로 말뭉치 (corpus)를 나누는 작업.
- 토큰 (Token): 가장 작은 단위**, 문맥에 따라서, **단어** (word), **형태소** (morpheme), 서브워드 (subword).

(1) 단어 토큰화 (Word Tokenization)

단어외에 단어구, 의미를 갖는 문자열 등

입력: Time is an illusion. Lunchtime double so!



- 구두점 (punctuation): ".", "!", ",", "?" 등 제외
- 띄어쓰기로 자르기

출력: "Time", "is", "an", "illusion", "Lunchtime", "double", "so"

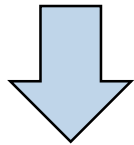
토큰화 (Tokenization)

- 토큰(Token)이라는 단위로 말뭉치 (corpus)를 나누는 작업.
- **구두점이나 특수 문자를 단순 제외해서는 안 된다.**

(1) 단어 토큰화 (Word Tokenization)

단어외에 단어구, 의미를 갖는 문자열 등

입력: Professor Sung has a Ph.D in A. I.



1. 구두점 (punctuation): ".", "!", ":", "?" 등 제외
2. 띄어쓰기로 자르기

출력: " Professor ", " Sung ", " has ", " a ", " Ph ", "D", "A", "I"

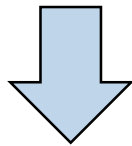
토큰화 (Tokenization)

- 토큰(Token)이라는 단위로 말뭉치 (corpus)를 나누는 작업.
- 구두점이나 특수 문자를 단순 제외해서는 안 된다.

(1) 단어 토큰화 (Word Tokenization)

단어외에 단어구, 의미를 갖는 문자열 등

입력: Professor Sung has a creative thinking ability. The ~~다른 문장



1. 구두점 (punctuation): ".", "!", ",", "?" 등 제외
2. 띄어쓰기로 자르기

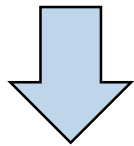
- 이 예제에서 마침표(.)와 같은 경우는 문장의 경계
- \$: \$45.55
- 날짜: 01/02/06
- 콤마 (,) : 123,456,789
- 띄어 쓰기: New York

토큰화 (Tokenization)

- 토큰(Token)이라는 단위로 말뭉치 (corpus)를 나누는 작업.
- **구두점이나 특수 문자를 단순 제외해서는 안 된다.**

(2) 문장 토큰화 (Sentence Tokenization)

입력: 애들이 이해했다고 해줘. 그리고 나서 수업 끝내자.



1. 구두점 (punctuation): ".", "!", ",", "?" 등 사용
- 2.

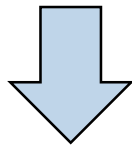
출력: " 애들이 이해했다고 해줘.", "그리고 나서 수업 끝내자. "

토큰화 (Tokenization)

- 토큰(Token)이라는 단위로 말뭉치 (corpus)를 나누는 작업.
- 구두점이나 특수 문자를 단순히 제외해서는 안 된다.

(2) 문장 토큰화 (Sentence Tokenization)

입력: 애들아 A. I. 수업 이해했다고 해줘. 그리고나서 수업 끝내자.



출력: " 애들아 A.", "I.", "이해했다고 해줘.", "그리고나서 수업 끝내자. "

- 사용하는 코퍼스가 어떤 국적의 언어인지, 또는 해당 코퍼스 내에서 특수문자들이 어떻게 사용되고 있는지에 따라서 직접 규칙들을 정의.
- 텍스트 전처리란 굉장히 어려운 작업.

한국어 토큰화 (Tokenization)

- 토큰(Token)이라는 단위로 말뭉치 (corpus)를 나누는 작업.
- 토큰 (Token): 가장 작은 단위**, 문맥에 따라서, **단어** (word), **형태소** (morpheme), 서브워드 (subword).

영어에서 **띄어쓰기** 기반 단어 토큰화

입력: Time is an illusion.



New York 같은 합성을 제외하면

출력: "Time", "is", "an", "illusion"

한국어에서 **띄어쓰기** (=어절) 기반 단어 토큰화 -> 지양되고 있음

입력: 성호가 자연어처리 수업을 한다.



출력: " 성호가 ", " 자연어처리", " 수업을 ", " 한다."

한국어 토큰화 (Tokenization)

- (3) 형태소 (morpheme) 기반 토큰화

한국어에서 띄어쓰기 (=어절) 기반 단어 토큰화 -> 지양되고 있음

입력: 성호가 자연어처리 수업을 한다.



출력: " 성호가 ", " 자연어처리", " 수업을 ", " 한다."

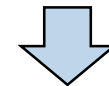
- 교착어=첨가어 (조사나 어미가 붙음)

성호는 = Sung is

성호와 = with Sung

성호에게 = to Sung

- 띄어쓰기 바로 없이 조사가 (는,와,에 게, 를, 가 등) 붙음
-> 같은 단어가 다른 단어로 인식

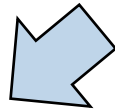


형태소 (morpheme) 개념

한국어 토큰화 (Tokenization)

- **형태소 (morpheme)**: 일정한 **의미가 (어휘+문법)** 있는 가장 작은 말의 단위
- **자립 형태소** : 접사, 어미, 조사와 상관없이 자립하여 사용할 수 있는 형태소. 그 자체로 단어가 된다. 체언(명사, 대명사, 수사), 수식언(관형사, 부사), 감탄사 등이 있다.
- **의존 형태소** : 다른 형태소와 결합하여 사용되는 형태소. 접사, 어미, 조사, 어간를 말한다.

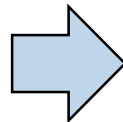
“성호가 자연어처리 수업을 한다.”



자립: 성호, 자연어, 처리, 수업

의존: -가, 을-, 하-, -ㄴ 다

(읽-, -었, -다,)



water, tree, for, the, when , to, for

kindness-> -ness, un-, -ful, re-

한국어 토큰화 (Tokenization)

- 형태소 (morpheme): 일정한 의미가 (어휘+문법) 있는 가장 작은 말의 단위
- 의존 형태소 : 다른 형태소와 결합하여 사용되는 형태소. 어간, 어미, 접사, 조사를 말한다.
- 어간/어미, 어근/접사
 - 어간 & 어미: 동사와 형용사에서 사용
 - (1) 동사: 먹다, 먹고, 먹니, 먹지, 먹으며, 먹어서
 - (2) 형용사: 예쁘다, 예쁘고, 예쁘니, 예쁘지
 - 접사(접두사): 햇과일, 맨손, 군살, 짓누르다, 치솟다
 - (어근=실질적 의미: 과일, 손, 살, 누르다, 솟다)
 - 접미사: 나무꾼, 선생님
 - (어근=실질적 의미: 나무, 선생)
 - 조사=명사에 붙는: 은,는,이,가, 를,을, 에게...

한국어 토큰화 (Tokenization)

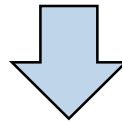
- 형태소 (morpheme): 일정한 의미가 (어휘+문법) 있는 가장 작은 말의 단위

어간과 어미, 어근과 접사			
동사와 형용사에서 사용하는 개념	어간	용언 활용 시 변하지 않는 부분	'덧붙이다'에서 '덧붙이-'
	어미	용언 활용 시 변하는 부분	'덧붙이다'에서 '-다'
모든 단어에서 쓸 수 있는 개념	어근	단어의 중심 의미를 나타내는 부분	'덧붙이다'에서 '붙-'
	접사	단어에 붙어 그 뜻을 제한하거나 문법적 기능을 하는 형식 형태소	'덧붙이다'에서 '덧-, -이'

한국어 토큰화 (Tokenization)

- **형태소 (morpheme):** 일정한 **의미가 (어휘+문법)** 있는 가장 작은 말의 단위
- **자립 형태소 :** 접사, 어미, 조사와 상관없이 자립하여 사용할 수 있는 형태소. 그 자체로 단어가 된다. 체언(명사, 대명사, 수사), 수식언(관형사, 부사), 감탄사 등이 있다.
- **의존 형태소 :** 다른 형태소와 결합하여 사용되는 형태소. 접사, 어미, 조사, 어간를 말한다.

“성호가 자연어처리 수업을 한다.”

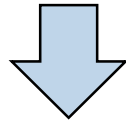


자립: 성호, 자연어, 처리, 수업

의존: -가(조사), 을-(조사), 하-(어간), -ㄴ 다(어미)

한국어 토큰화 (Tokenization)

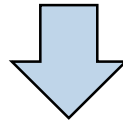
“성호가 자연어처리 수업을 한다.”



형태소 (morpheme):

자립: 성호, 자연어, 처리, 수업

의존: -가(조사), 을-(조사), 하-(어간), -ㄴ 다(어미)



Vector 생성

성호	-가	자연어	처리	수업	을-	하-	-ㄴ 다

지난 시간 리뷰

- 전처리 – 토큰화
- 토큰 (Token): 가장 작은 단위, 문맥에 따라서, 단어 (word), 형태소 (morpheme)

영어-띄어쓰기 단어

입력: Time is an illusion. Lunchtime double so!

1. 구두점 (punctuation): ".", "!", ":", "?" 등 제외
2. 띄어쓰기로 자르기

출력: "Time", "is", "an", "illusion", "Lunchtime", "double", "so"

∴ 형태소 ≡ 단어

단어: 한다

형태소: 하- (어간), -ㄴ 다(어미)

한국어 -형태소

“성호가 자연어처리 수업을 한다.”



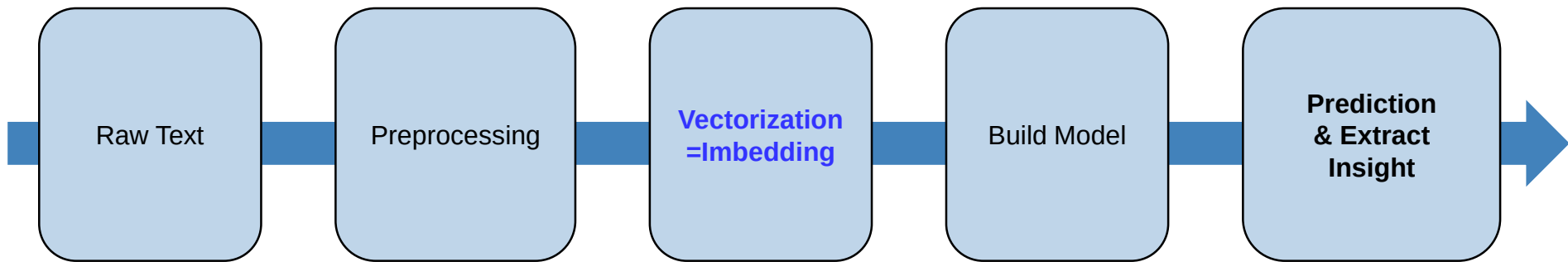
자립: 성호, 자연어, 처리, 수업

의존: -가(조사), 을-(조사), 하-(어간), -ㄴ 다(어미)

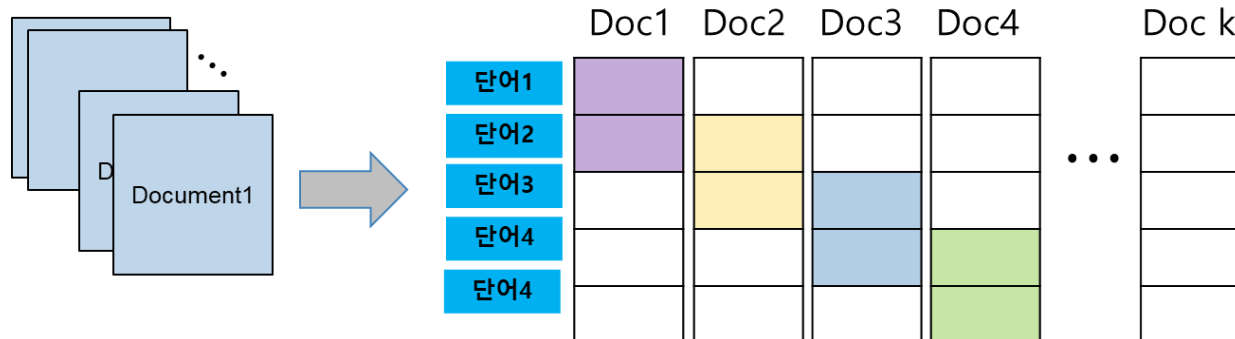
- 어간 & 어미: 동사와 형용사에서 사용
- 접사(접두사): **햇**과일, **맨**손, **군**살, **짓**누르다, **치**솟다
 - (어근=실질적 의미: 과일, 손, 살, 누르다, 솟다)
- 접미사: 나무**꾼**, 선생**님**
 - (어근=실질적 의미: 나무, 선생)
- 조사=명사에 붙는: 은,는,이,가, 를,을, 에게...

지난 시간 리뷰

Text Analytics Processing



From unstructured to structured



지난 시간 리뷰

- 전처리 – 토큰화
- 토큰 (Token): 가장 작은 단위, 문맥에 따라서, 단어 (word), 형태소 (morpheme)

“성호가 자연어처리 수업을 한다.”



자립: 성호, 자연어, 처리, 수업

의존: -가(조사), 을-(조사), 하-(어간), -ㄴ 다(어미)



Vector 생성

성호	-가	자연 어	처리	수업	을-	하-	-ㄴ 다

품사 태깅 (Part of Speech [POS] Tagging)

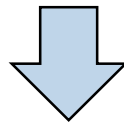
```
Out[4]:
```

	id	document	label
0	8112052	머릴때보고 지금다시봐도 재밌어요 ㅋㅋ	1
1	8132799	디자인을 배우는 학생으로, 외국디자이너와 그들이 일군 전통을 통해 발전해가는 문화산...	1
2	4655635	폴리스스토리 시리즈는 1부터 뉴까지 버릴때 하나도 없음.. 최고.	1
3	9251303	와.. 연기가 진짜 개쩔구나.. 지루할거라고 생각했는데 몰입해서 봤다.. 그래 이런...	1
4	10067386	안개 자욱한 밤하늘에 떠 있는 초승달 같은 영화.	1

```
kkma.morphs(temp_data[0])
```

['어리', 'ㄹ', '때', '보고', '지금', '다시', '보', '아도', '재밌', '어요', 'ㅋㅋ']

형용사



High Dimension = 고차원 이슈

```
In [93]: kkma.morphs(temp_data[2])
```

```
Out[93]:
```

['폴리스',
'스토리',
'시리즈',
'는',
'1',
'부터',
'뉴',
'까',
'지',
'버리',
'ㄹ게',
'하나',
'도',
'없',
'음',
'..',
'최고',
'']

Unnamed: 0	어리	ㄹ	때	보고	지금	다시	보	아도	재밌	...	뉴	까	지	버리	ㄹ게	하나	없	음	..	최고
0	0	1	1	1	1	1	1	1	1	1	...	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0
2	2	0	0	0	0	0	0	0	0	0	...	1	1	1	1	1	1	1	1	1

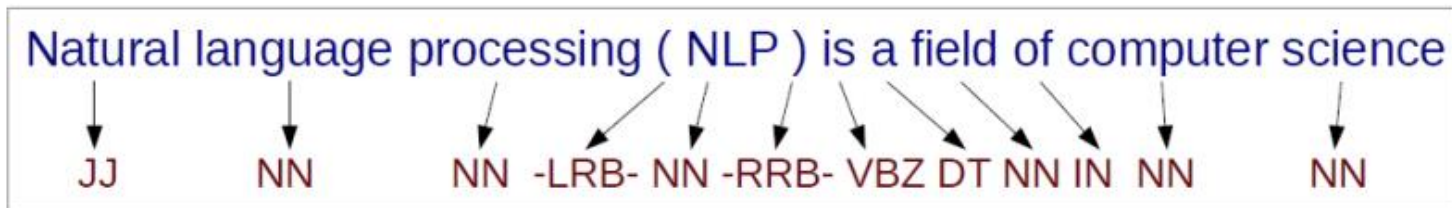
품사 태깅 (Part of Speech [POS] Tagging)

- 품사 (단어의 종류) 태깅이란 형태소 단위 분리 후, 해당 **형태소의 품사를 태깅하는 것을 의미.**
- Input: tokens that sentence may have **ambiguity**
- output: most appropriate tag by considering its definition and contexts

형용사

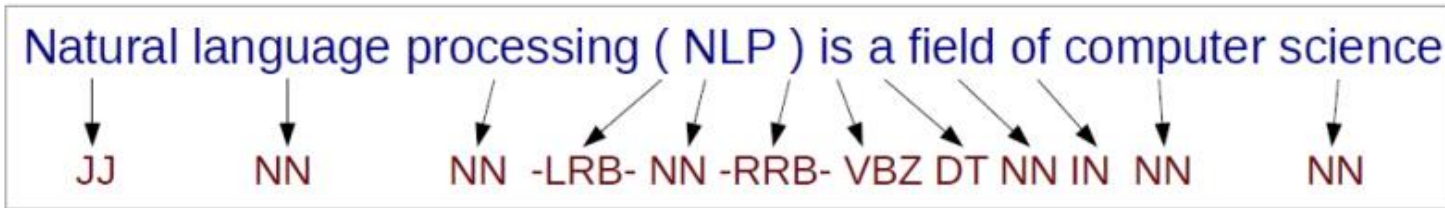
명사

동사



- Different POS tags for the same token
 - ✓ I love you. → “love” is a **verb**
 - ✓ All you need is love. → “love” is **noun**

품사 태깅 (Part of Speech [POS] Tagging)



<https://wayne.tistory.com/695>

DT	한정사
QT	수량사
CD	기수
NN	명사, 단수형
NNS	명사, 복수형
NNP	고유 명사, 단수형
NNPS	고유 명사, 복수형
EX	<i>There was a party</i> 구문에서와 같이, 존재 <i>there</i> .
PRP	인칭 대명사(PP)
PRP\$	소유대명사(PP\$)
POS	소유 종료
RBS	부사, 최상급
RBR	부사, 비교급
RB	부사
JJS	형용사, 최상급
JJR	형용사, 비교급
JJ	형용사
MD	법조동사
VB	동사, 기본형
VBP	동사, 현재형 시제, 3인칭 단수 제외
VBZ	동사, 현재형 시제, 3인칭 단수

품사 태깅 (Part of Speech [POS] Tagging)

- 품사 (단어의 종류) 태깅이란 형태소 단위 분리 후, 해당 **형태소의 품사를 태깅하는 것을 의미.**
- 명사대명사, 수사 (체언)
- 관형사, 부사, 조사, 감탄사
- 동사, 형용사 (용언)

꼬꼬마 품사 태깅 표

대분류	세종 품사 태그		심광선 품사 태그		KKMA 단일 태그 V 1.0				
	태그	설명	Class	설명	목록 1	목록 2	태그	설명	활용태그
체언	NNG	일반 명사	NN	명사	N	NN	NNG	보통 명사	NNA
	NNP	고유 명사					NNP	고유 명사	
	NNB	의존 명사	NX	의존 명사			NNB	일반 의존 명사	NNB
	NR	수사	NU	수사			NR	수사	NR
	NP	대명사	NP	대명사			NP	대명사	NP
용언	VV	동사	VV	동사	V	VV	VV	동사	VV
	VA	형용사	AJ	형용사			VA	형용사	VA
	VX	보조 용언	VX	보조 동사			VXV	보조 동사	VX
	VX	보조 용언	AX	보조 형용사			VXA	보조 형용사	VX
	VCP	긍정 지정사	CP	서술격 조사 '이다'		VC	VCP	긍정 지정사, 서술격 조사 '이다'	VCP
	VCN	부정 지정사					VCN	부정 지정사, 형용사 '아니다'	VCN
관형사	MM	관형사	DT	일반 관형사	M	MD	MDT	일반 관형사	MD
			DN	수 관형사			MDN	수 관형사	MD
부사	MAG	일반 부사	AD	부사		MA	MAG	일반 부사	MAG
	MAJ	접속 부사					MAC	접속 부사	MAC
감탄사	IC	감탄사	EX	감탄사	I	IC	IC	감탄사	IC
조사	JKS	주격 조사	JO	조사	J	JK	JKS	주격 조사	JKS
	JKC	보격 조사					JKC	보격 조사	JKC
	JKG	관형격 조사					JKG	관형격 조사	JKG
	JKO	목적격 조사					JKO	목적격 조사	JKO
	JKB	부사격 조사					JKM	부사격 조사	JKM
	JKV	호격 조사					JKI	호격 조사	JKI
	JKQ	인용격 조사					JKQ	인용격 조사	JKQ
	JX	보조사				JX	JX	보조사	JX
	JC	접속 조사				JC	JC	접속 조사	JC

선어말 어미	EP	선어말 어미	EP	선어말 어미	EP	EPH 존칭 선어말 어미	EP	raw.dic	
						EPT 시제 선어말 어미			
어말 어미	EF	종결 어미	EM	어말 어미	E	EPP 공손 선어말 어미	EF		
						EFN 명사형 종결 어미			
						EFQ 의문형 종결 어미			
						EFO 명령형 종결 어미			
						EFA 청유형 종결 어미			
						EFI 감탄형 종결 어미			
						EFR 존칭형 종결 어미			
	EO	연결 어미				ECE 대동 연결 어미	EO		
						EOD 의존적 연결 어미			
						ECS 보조적 연결 어미			
ETN 명사형 전성 어미						ETN			
ETM 관형형 전성 어미	ETD								
접두사	XPN 체언 접두사	PF	접두사	XP	XPN 체언 접두사	XPN			
					XPV 용언 접두사	XPV			
접미사	XSN 명사 파생 접미사	SN	명사화 접미사	XS	XSN 명사 파생 접미사	XSN	simple.dic		
	XSV 동사 파생 접미사	SV	동사화 접미사		XSV 동사 파생 접미사	XSV			
	XSA 형용사 파생 접미사	SJ	형용사화 접미사		XSA 형용사 파생 접미사	XSA			
		SA	부사화 접미사		※※※ 부사 파생 접미사	※※※			
		SF	기타 접미사		※※※ 기타 접미사	※※※			
어근	XR	어근	XR	XR	XR	어근	XR		
부호	9F 마침표, 줄표, 느낌표	SY	부호 외래어	S	9F	9F	마침표, 줄표, 느낌표	9F	Symbol class
	9P 원표, 가운뎃점, 물결표, 빗줄				9P	9P	원표, 가운뎃점, 물결표, 빗줄	9P	
	9S 따옴표, 앞표, 뒤표				9S	9S	따옴표, 앞표, 뒤표	9S	
	9E 줄임표				9E	9E	줄임표	9E	
	9O 물결표 (물결, 슬결, 바결)				9O	9O	물결표 (물결, 슬결, 바결)	9O	
	9W 기타기호 (논리수학기호, 화폐기호)				9W	9W	기타기호 (논리수학기호, 화폐기호)	9W	
분석 능력	NF 명사추정범주	NR	미동특어	U	UN	UN	명사추정범주	NNA	N/A
	NV 용언추정범주				UV	UV	용언추정범주	N/A	
	NA 분석불능범주				UE	UE	분석불능범주	N/A	
	9L 외래어				OL	OL	외래어	NNA	
한글 이외	9H 한자			O	OH	OH	한자	NNA	
	9N 숫자				ON	ON	숫자	NR	

품사 태깅 (Part of Speech [POS] Tagging)

- Why?

- 모호성 Ambiguity

단어	POS 태깅 결과
1 the manufacture of cars	the/DT manufacture/NN of/IN cars/NNS
2 We manufacture memory products	We/PRP manufacture/VBP memory/NN products/NNS
3 manufacture steel products	manufacture/NN steel/NN products/NNS

- Different POS tags for the same token

✓ I love you. → “love” is a **verb**

✓ All you need is love. → “love” is **noun**

- Dimension Reduction, Feature Reduction (명사, 형용사)

그	영화	...	재미	있다

품사 태깅 (Part of Speech [POS] Tagging)

- POS Tagging Algorithms

Fundamentals

POS-Tagging generally requires:

Training phase where a **manually annotated** corpus is processed by a machine learning algorithm; and a

Tagging algorithm that processes texts using learned parameters.

Performance is generally good (around 96%) when staying in the same domain.

Algorithms used in POS-Tagging

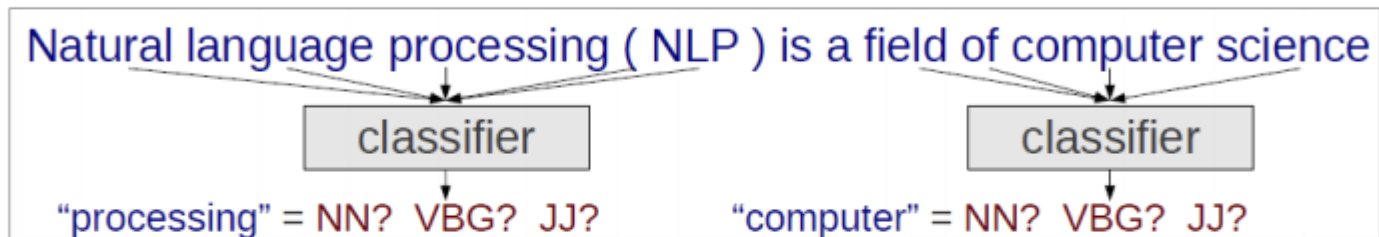
There is a multitude of approaches, commonly used are:

- Decision Trees
- Hidden Markov Models (HMMs)
- Support Vector Machines (SVM)
- Transformation-based Taggers (e.g., the **Brill** tagger)

품사 태깅 (Part of Speech [POS] Tagging)

- POS Tagging Algorithms

- ✓ **Pointwise prediction**: predict each word individually with a classifier (e.g. Maximum Entropy Model, SVM)

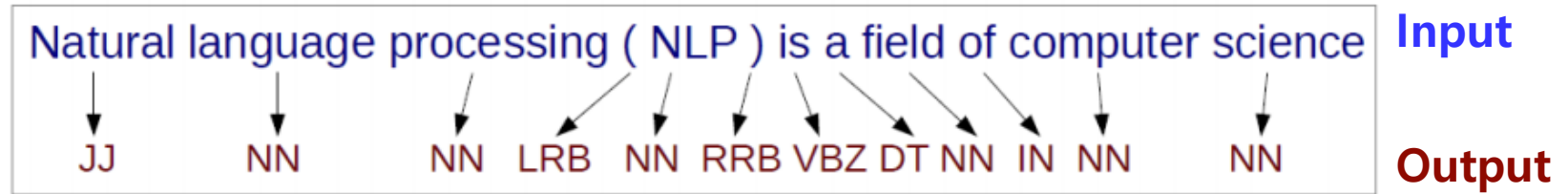


- ✓ Probabilistic models

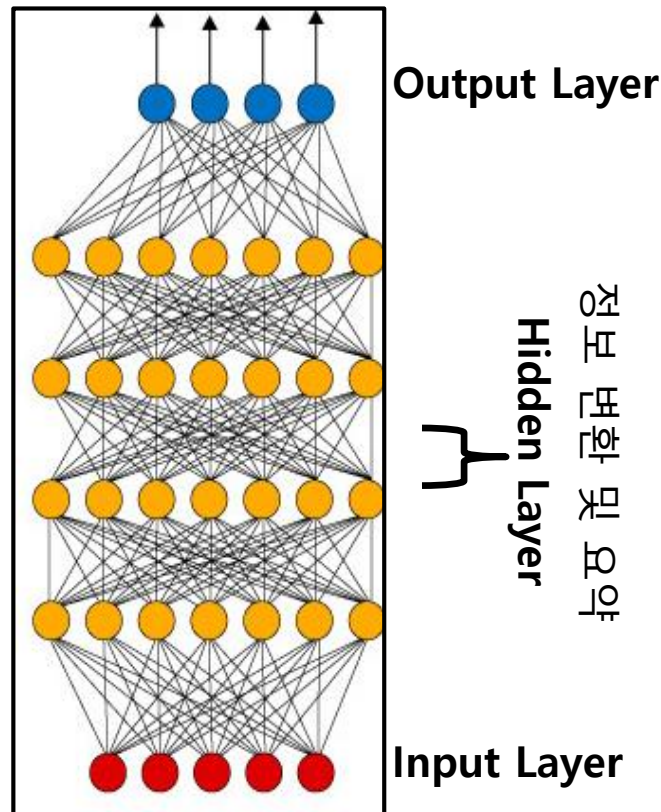
- **Generative sequence models**: Find the most probable tag sequence given the sentence (Hidden Markov Model; HMM)
- **Discriminative sequence models**: Predict whole sequence with a classifier (Conditional Random Field; CRF)

- ✓ Neural network-based models

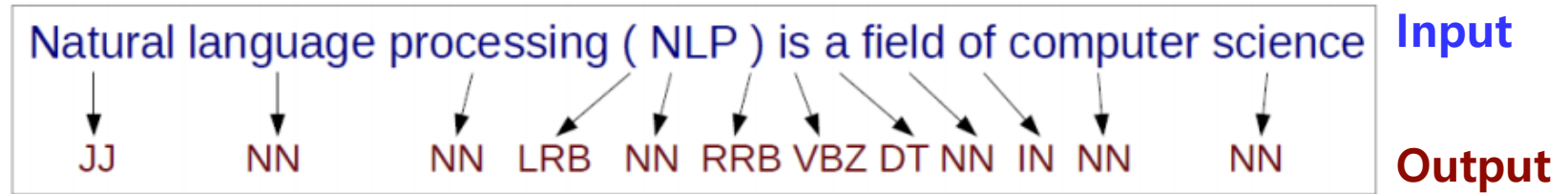
품사 태깅 (Part of Speech [POS] Tagging)



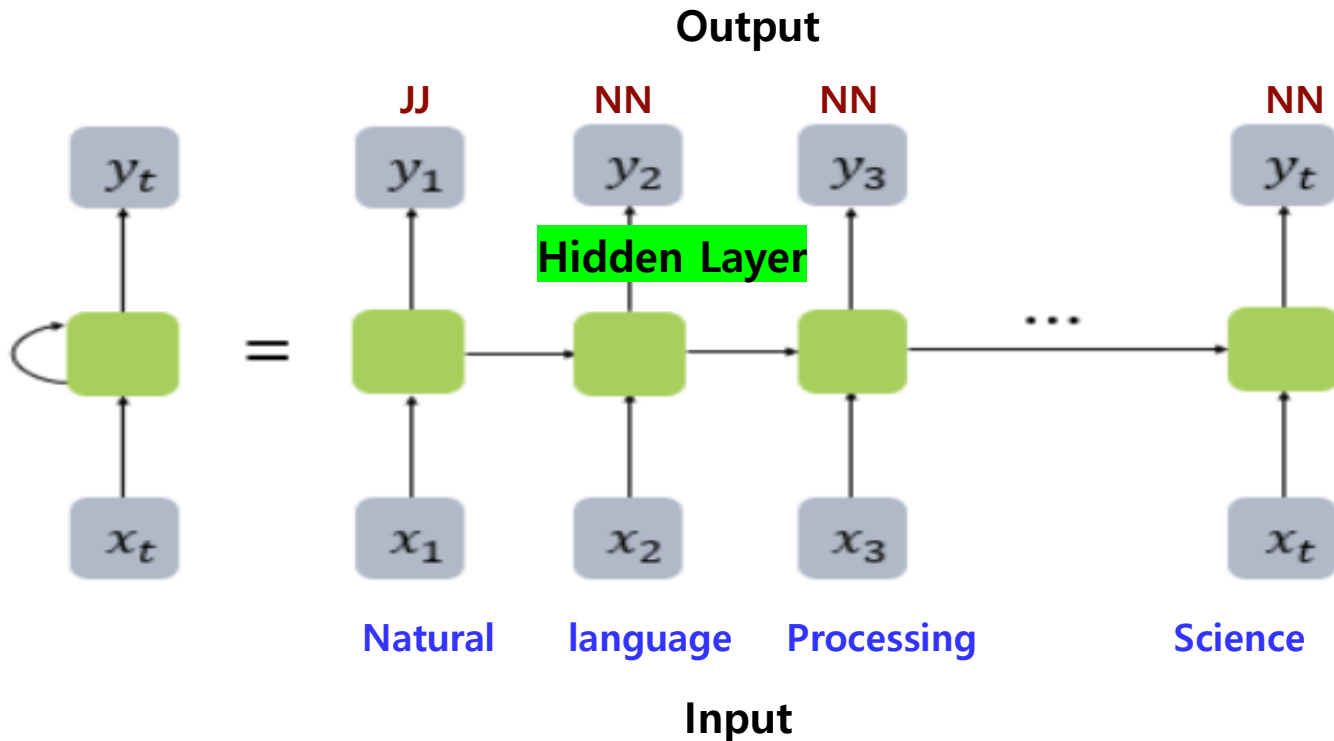
Feed Forward Neural Networks



품사 태깅 (Part of Speech [POS] Tagging)



Recurrent Neural Networks (RNN): 순환신경망



현재 시점 t 에서의 은닉층 (메모리 셀)이 갖고 있는 값은 과거의 은닉층값에 영향을 받은 것임

형태 분석 (Morphological Analysis)

- 어간추출 (Stemming), 표제어 추출 (Lemmatization)

말 뭉치 corpus

```
Out[4]:
      id      document label
0  8112052      어릴때보고 지금다시봐도 재밌어요ㅋㅋ      1
1  8132799  디자인을 배우는 학생으로, 외국디자인과 그들이 일군 전통을 통해 발전해가는 문화산...      1
2  4655635  폴리스톰리 시리즈는 1부터 뉴까지 버릴때 하나도 없음.. 최고.      1
3  9251303  와.. 연기가 진짜 개쩔구나.. 지루할거라고 생각했는데 몰입해서 봤다.. 그래 이런...      1
4  10067386  안개 자욱한 밤하늘에 떠 있는 초승달 같은 영화.      1
```



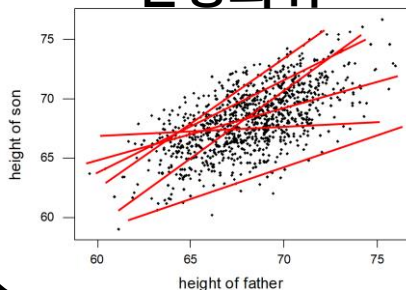
High-dimensional Vector 생성

성호	-가	자연 어	처리	수업	을-	하-	-ㄴ 다	...	...



Machine Learning Models

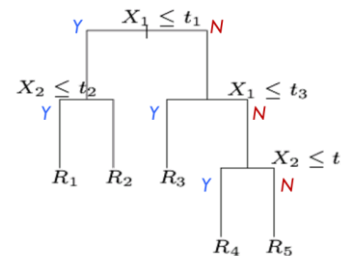
선형회귀



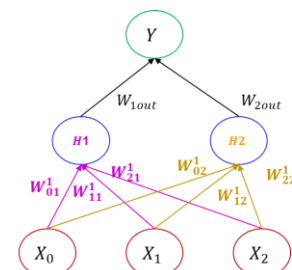
앙상블



의사결정



신경망



형태 분석 (Morphological Analysis)

- 어간추출 (Stemming), 표제어 추출 (Lemmatization)

- 문법적으로 다양한 형태의 변종을 단순화 시키는 작업
- 의미를 최소한 훼손시키는 않는 범위 내에서 차원 축소
- 정규화 작업 중 하나
- **Morphological Analysis**은 형태소로 부터 출발

Morphological Variants

Words are changed through a morphological process called *inflection*:

- typically indicates changes in case, gender, number, tense, etc.
- example *car* → *cars*, *give* → *gives*, *gave*, *given*

Goal: “normalize” words

Stemming and Lemmatization

Two main approaches to normalization:

Stemming reduce words to a *base form*

Lemmatization reduce words to their *lemma*

Main difference: stemming just finds **any** base form, which doesn't even need to be a word in the language! Lemmatization find the actual *root* of a word, but requires morphological analysis.

형태 분석 (Morphological Analysis)

- 어간추출 (Stemming), 표제어 추출 (Lemmatization)

- Stemming

Stemming

Commonly used in Information Retrieval:

- Can be achieved with rule-based algorithms, usually based on suffix-stripping **'-ed'**
- Standard algorithm for English: the *Porter* stemmer
- Advantages: simple & fast
- Disadvantages:
 - Rules are language-dependent
 - Can create words that do not exist in the language, e.g.,
computers → *comput*
 - Often reduces different words to the same stem, e.g.,
army, arm → *arm*
stocks, stockings → *stock*
- Stemming for German: German stemmer in the full-text search engine *Lucene*, *Snowball* stemmer with German rule file

형태 분석 (Morphological Analysis)

- 어간추출 (Stemming), 표제어 추출 (Lemmatization)

- Lemmatization

Lemmatization

Lemmatization is the process of deriving the base form, or *lemma*, of a word from one of its inflected forms. This requires a morphological analysis, which in turn typically requires a *lexicon*.

- Advantages:
 - identifies the *lemma* (root form), which is an actual word
 - less errors than in stemming **am, are, is -> be**
- Disadvantages:
 - more complex than stemming, slower
 - requires additional language-dependent resources
- While stemming is good enough for Information Retrieval, Text Mining often requires lemmatization
 - Semantics is more important (we need to distinguish an *army* and an *arm*!)
 - Errors in low-level components can multiply when running downstream

형태 분석 (Morphological Analysis)

- 어간추출 (Stemming), 표제어 추출 (Lemmatization)

	Word	Stemming	Lemmatization
	Love	Lov	Love
	Loves	Lov	Love
	Loved	Lov	Love
	Loving	Lov	Love
명사	Innovation	Innovat	Innovation
	Innovations	Innovat	Innovation
동사	Innovate	Innovat	Innovate
	Innovates	Innovat	Innovate
형용사	Innovative	Innovat	Innovative

품사 반영 x
품사 반영 o

- 데이터의 차원 축소 관점에서 효과 -> 모델 학습이 쉽고, 모델학습 비용 감소.
- 의미론적 (Semantics) 관점에서 중요도가 떨어짐.
- Lemmatization의 경우 POS 태그를 고려

형태 분석 (Morphological Analysis)

- 어간추출 (Stemming), 표제어 추출 (Lemmatization)

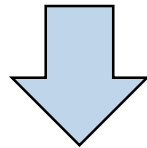
- 형태소 분석을 기반으로 함
- 한국어의 경우 POS TAGGING 표제어 및 어간 추출을 비슷하게 대신 함

```
#####  
#한국어 lemma stemming #####  
#####  
  
text1="했다"  
  
text2="했었다"  
  
  
print(kkma.pos(text1))  
print(kkma.pos(text2))
```

7. 어간 '하-' 뒤에 어미 '-아'가 결합하여 '하여'로 바뀌어 나타나는 경우, 바뀐 대로 적는다. '하여'는 '해'로 줄어 들 수 있다.

(한글 맞춤법 제34항 붙임 2 참조)

하다 하--아→하여 →해
하--아라→하여라→해라
하--아도→하여도→해도
하--았다→하였다→했다



동사(어간)

```
...: print(kkma.pos(text1))  
...: print(kkma.pos(text2))  
[('하', 'VV'), ('었', 'EPT'), ('다', 'EFN')]  
[('하', 'VV'), ('었', 'EPT'), ('었', 'EPT'), ('다', 'EFN')]
```

불용어 (Stopword) 처리

- 유의미한 토큰만을 선별하기 위한 작업
- 예: I, my, me, over, 조사, 접미사
- 한국어 예: '의','가','이','은','들','는','좀','잘','과','도','를','으로','자','에','와','한','하다']

Korean Stopwords

아	어찌됐든	하기보다는
휴	그위에	차라리
아이구	게다가	하는 편이 낫다
아이쿠	점에서 보아	호호
아이고	비추어 보아	놀라다
어	고려하면	상대적으로 말하
나	하게될것이다	자면
우리	일것이다	마치
저희	비교적	아니라면
따라	좀	췌
의해	보다더	그렇지 않으면
을	비하면	그렇지 않다면
를	시키다	안 그러면
에	하게하다	아니었다면
의	할만하다	하든지
가	의해서	아니면
으로	연이서	이라면
로	이어서	좋아
에게	잇따라	알았어
뿐이다	뒤따라	하는것도
의거하여	뒤이어	그만이다
근거하여	결국	어쩔수 없다
입각하여	의지하여	하나
기준으로	기대여	일
예하면	통하여	일반적으로

- 조사나 접속사와 같은 단어들 뿐만 아니라 명사, 형용사와 같은 단어들 중에서 불용어로서 제거
- 사용자가 직접 불용어 사전을 정의하는 경우가 많음

정규 표현식 (Regular Expression)

파이썬 실습

특정한 패턴과 일치하는 문자열을 '검색', '치환', '제거' 하는 기능을 지원

특수 문자	설명
.	한 개의 임의의 문자를 나타냅니다. (줄바꿈 문자인 <code>\n</code> 는 제외)
?	앞의 문자가 존재할 수도 있고, 존재하지 않을 수도 있습니다. (문자가 0개 또는 1개)
*	앞의 문자가 무한개로 존재할 수도 있고, 존재하지 않을 수도 있습니다. (문자가 0개 이상)
+	앞의 문자가 최소 한 개 이상 존재합니다. (문자가 1개 이상)
^	뒤의 문자로 문자열이 시작됩니다.
\$	앞의 문자로 문자열이 끝납니다.
{숫자}	숫자만큼 반복합니다.
{숫자1, 숫자2}	숫자1 이상 숫자2 이하만큼 반복합니다. <code>?</code> , <code>*</code> , <code>+</code> 를 이것으로 대체할 수 있습니다.
{숫자,}	숫자 이상만큼 반복합니다.
[]	대괄호 안의 문자들 중 한 개의 문자와 매치합니다. <code>[amk]</code> 라고 한다면 <code>a</code> 또는 <code>m</code> 또는 <code>k</code> 중 하나라도 존재하면 매치를 의미합니다. <code>[a-z]</code> 와 같이 범위를 지정할 수도 있습니다. <code>[a-zA-Z]</code> 는 알파벳 전체를 의미하는 범위이며, 문자열에 알파벳이 존재하면 매치를 의미합니다.
[^문자]	해당 문자를 제외한 문자를 매치합니다.
	A B와 같이 쓰이며 A 또는 B의 의미를 가집니다.

문자 규칙	설명
<code>\w</code>	역 슬래쉬 문자 자체를 의미합니다
<code>\wd</code>	모든 숫자를 의미합니다. <code>[0-9]</code> 와 의미가 동일합니다.
<code>\WD</code>	숫자를 제외한 모든 문자를 의미합니다. <code>[\^0-9]</code> 와 의미가 동일합니다.
<code>\ws</code>	공백을 의미합니다. <code>[\t\n\r\f\v]</code> 와 의미가 동일합니다.
<code>\WS</code>	공백을 제외한 문자를 의미합니다. <code>[\^ \t\n\r\f\v]</code> 와 의미가 동일합니다.
<code>\ww</code>	문자 또는 숫자를 의미합니다. <code>[a-zA-Z0-9]</code> 와 의미가 동일합니다.
<code>\WW</code>	문자 또는 숫자가 아닌 문자를 의미합니다. <code>[\^a-zA-Z0-9]</code> 와 의미가 동일합니다.
모듈 함수	설명
<code>re.compile()</code>	정규표현식을 컴파일하는 함수입니다. 다시 말해, 파이썬에게 전해주는 역할을 합니다. 찾고자 하는 패턴이 빈번한 경우에는 미리 컴파일해놓고 사용하면 속도와 편의성면에서 유리합니다.
<code>re.search()</code>	문자열 전체에 대해서 정규표현식과 매치되는지를 검색합니다.
<code>re.match()</code>	문자열의 처음이 정규표현식과 매치되는지를 검색합니다.
<code>re.split()</code>	정규 표현식을 기준으로 문자열을 분리하여 리스트로 리턴합니다.
<code>re.findall()</code>	문자열에서 정규 표현식과 매치되는 모든 경우의 문자열을 찾아서 리스트로 리턴합니다. 만약, 매치되는 문자열이 없다면 빈 리스트가 리턴됩니다.
<code>re.finditer()</code>	문자열에서 정규 표현식과 매치되는 모든 경우의 문자열에 대한 이터레이터 객체를 리턴합니다.
<code>re.sub()</code>	문자열에서 정규 표현식과 일치하는 부분에 대해서 다른 문자열로 대체합니다.

품사 태깅 (Part of Speech [POS] Tagging)

오픈소스 한국어 형태소 분석기

한나눔 (Hananum)

- 명칭: 한나눔
- 관리: Kaist
- 개발언어: Java
- 알고리즘: HMM
- 학습사전: 세종계획 산출물
- 소스 오픈 여부: 오픈
- 구분분석기: 없음

꼬꼬마 (KKMA)

- 명칭: 꼬꼬마
- 관리: Seoul University
- 개발언어: Java
- 알고리즘: ?
- 학습사전: 세종계획 산출물
- 소스 오픈 여부: Open 별도 요청
- 구분분석기: 일부 탑재

메카브 KO (Mecab)

- 명칭: 메카브 KO (Mecab), 은전한닢
- 관리: 은전한닢 프로젝트 커뮤니티
- 개발언어: C/C++
- 알고리즘: CRF
- 학습사전: 세종계획 산출물 (일부)
- 소스 오픈 여부: 오픈
- 구분분석기: 없음

https://intothedata.com/02.scholar_category/natural_language_processing/korean_pos_tagger/